第2章 计算机中的数和数制

西南交通大学计算机与人工智能学院

keli@home.swjtu.edu.cn

本章主要内容

1. 数和数制
2. 带符号二进制数的表示和运算
3. 二进制编码的十进制数
4. 无符号整数
5. 字符的表示法

2.1 数和数制

2.1.1 数制的表示

- 数制也称为计数制
- 是指用一组固定的数字符号和统一的规则表示数的方法。
- 对于任意r进制数，可以用下式表示：

$$\sum_{i=-m}^{n} a_i r^i = a_{-m} r^{-m} + \cdots + a_{-2} r^{-2} + a_{-1} r^{-1} + a_0 r^0 + a_1 r^1 + \cdots + a_n r^n$$

1. 二进制

只有0和1两个数字符号，“逢二进一，借一当二”。

1 1 0 1 . 0 1 $= 1\times2^3 + 1\times2^2 + 0 + 1\times2^0 + 0+1\times2^{-2}$

2^3 2^2 2^1 2^0 2^{-1} 2^{-2}

2. 八进制与十六进制（用于阅读和书写）

八进制：数码有八个：0，1，2，3，4，5，6，7 .

逢八进一，借一当八

十六进制：数码有十六个：0, 1，…, 9，A, B, C, D, E, F

3．几个概念

权值/位权：某个固定位置上的计数单位。例：

1 1 0 1 . 0 1 $= 1*2^3+1*2^2+0+1*2^0+0+1*2^{-2}$

2^3 2^2 2^1 2^0 2^{-1} 2^{-2} 权值

基数：指某个进制的值R。

例如：二进制数的基数为2，十进制数的基数为10.

4．进制数的表示方式

1）用下标加以标注。例如：$(1010)_2$，$(1010)_{10}$

2）用后缀字母表示不同的进制。

B — 二进制

Q — 八进制

H — 十六进制

D — 十进制

例如：375Q，　0A17H，　101B

注：为了便于计算机识别，汇编程序规定，当十六进制数的首字符为字母时，前面加数字0。

常用几种进制的对应关系

十进制	二进制	八进制	十六进制
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

2.1.2 数制之间的转换

1. 其他数制转为十进制数：

方法：按权展开

每位数字乘以其权所得到的乘积之和即为其所表示的数的值。

2.1.2 数制之间的转换

【例2-1】将1010.101B、23.4Q和56.78H转换成十进制。

解：

$$1010.101B = 1\times2^3 + 0\times2^2 + 1\times2^1 + 0\times2^0 + 1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3} = 10.625D$$

$$23.4Q = 2\times8^1 + 3\times8^0 + 4\times8^{-1} = 19.5D$$

$$0FA3.4H = 15\times16^2 + 10\times16^1 + 3\times16^0 + 4\times16^{-1} = 4003.25D$$

2.1.2 数制之间的转换

2. 十进制数转换为其它数制数

- 降幂法

① 假设要转换的十进制数为N, 找出最接近N并小于等于N的r进制位权值r^i；

② $N=N-C\times r^i$， （$C\geqslant 0$且$N<r^i$）；

③ 记录这个C, 作为i位的位码a_i；

④ $i=i-1$，得到下一个位权值r^i；

⑤ 重复2、3、4 ，直至N为0或达到所需精度。

【例2-2】把十进制数117.8125转换成二进制数

N	C		r^i			a_i		
117	−1	×	2^6	=	53	（a_6	=1）	高位
53	−1	×	2^5	=	21	（a_5	=1）	
21	−1	×	2^4	=	5	（a_4	=1）	
5	−0	×	2^3	=	5	（a_3	=0）	
5	−1	×	2^2	=	1	（a_2	=1）	
1	−0	×	2^1	=	1	（a_1	=0）	
1	−1	×	2^0	=	0	（a_0	=1）	
0.8125	−1	×	2^{-1}	=	0.3125	（a_{-1}	=1）	
0.3125	−1	×	2^{-2}	=	0.0625	（a_{-2}	=1）	
0.0625	−0	×	2^{-3}	=	0.0625	（a_{-3}	=0）	
0.0625	−1	×	2^{-4}	=	0	（a_{-4}	=1）	低位

转换结果： 117.8125D=1110101.1101B

2.1.2 数制之间的转换

2. 十进制数转换为其它数制数

- 乘除法

① 整数部分：

除以基数取余，直至商为0；

② 小数部分：

乘以基数取整，直至积为整数或小数位数由精度定。

【例2-2】把十进制数117.8125转换成二进制数。

整数部分：

	商	余数	
117÷2＝	58	→ 1	整数部分低位
58÷2＝	29	→ 0	
29÷2＝	14	→ 1	
14÷2＝	7	→ 0	
7÷2＝	3	→ 1	
3÷2＝	1	→ 1	
1÷2＝	0	→ 1	整数部分高位

小数部分：

	积	整数	
0.8125×2＝	1.625	→ 1	小数部分高位
0.625 ×2＝	1.25	→ 1	
0.25 ×2＝	0.5	→ 0	
0.5 ×2＝	1.0	→ 1	小数部分低位

转换结果： 117.8125D=1110101.1101B

【例2-3】把十进制数48956转换成十六进制数

N	C		r^i			a_i	
48956	－11	×	16^3	＝	3900	（a_3＝B）	高位
3900	－15	×	16^2	＝	60	（a_2＝F）	↓
60	－3	×	16^1	＝	12	（a_1＝3）	
12	－12	×	16^0	＝	0	（a_0＝C）	低位

转换结果：48956D= BF3CH

2.1.2 数制之间的转换

3．二进制→八进制、十六进制的转换

二进制： 101110011.110 → 八进制：563.6_8

2^6 2^3 2^0 2^{-3}

即 8^2 8^1 8^0 8^{-1}

二进制： 101110011.1100 → 十六进制：173.C H

2^8 2^4 2^0 2^{-4}

即 16^2 16^1 16^0 16^{-1}

2.1.3 二进制数的运算

- 算术运算

① 加法规则：逢二进一。即：

0 + 0 = 0；　1 + 0 = 1；

0 + 1 = 1；　1 + 1 = 10；

② 减法规则：借一当二。即：

0 − 0 = 0；　1 − 0 = 1；

0 − 1 = 1；　1 − 1 = 0；

③ 乘法规则：任何数乘以0得0，1乘以任何数得该数。即：

0 × 0=0；　0 × 1=0；

1 × 0=0；　1 × 1=1；

④ 除法规则：0除以任何数得0，任何数除以1得该数，除数不得为0。即：

0÷1=0 ；　1÷1=1 。

2.1.3 二进制数的运算

逻辑运算

“与”运算（AND）

A	B	A∧B
0	0	0
0	1	0
1	0	0
1	1	1

“或”运算（OR）

A	B	A∨B
0	0	0
0	1	1
1	0	1
1	1	1

“非”运算（NOT）

A	A
0	1
1	0

“异或”运算（XOR）

A	B	A∀B
0	0	0
0	1	1
1	0	1
1	1	0

2.2带符号二进制数的表示与运算

- 数分为带符号数和无符号数两种类型；
- 带符号数有正数和负数之分。在计算机中，用0或1来表示数的正负。
- 在计算机中，一个数连同其符号在内的数值化的表示形式称为机器数。一般用最高位表示数的符号。而这个数本身的值称为真值。
- 机器数可以用不同编码方法表示。常用的编码方式有：原码、反码和补码。

2.2.1带符号二进制数的表示

1. 原码表示法：符号 + 绝对值

- 最高位表示符号（正数用0, 负数用1）
- 其他位表示数值位
- 数x的原码记作$[x]_{原}$，如机器字长为n，则原码的定义如下：

$$[x]_{原}=\begin{cases} x & 0 \leqslant x \leqslant 2^{n-1}-1 \\ 2^{n-1}+|x| & -(2^{n-1}-1) \leqslant x \leqslant 0 \end{cases}$$

1. 原码

例：当机器字长n＝8时，

$[+0D]_{原} = 0000\ 0000$，$[-0D]_{原} = 1000\ 0000$

$[+1D]_{原} = 0000\ 0001$，$[-1D]_{原} = 1000\ 0001$

$[+45D]_{原} = 0010\ 1101$，$[-45D]_{原} = 1010\ 1101$

$[+127D]_{原} = 0111\ 1111$，$[-127D]_{原} = 1111\ 1111$

∴ 0 的表示不惟一

1. 原码

- 按照定义，设n为字长，则原码能表示的整数范围是：$-(2^{n-1}-1)\sim+(2^{n-1}-1)$

例如：

- 8位二进制原码表示的整数范围是

  $-127\sim+127$

- 16位二进制原码表示的整数范围是

  $-32767\sim+32767$

2. 反码

反码表示法：

数x的反码记作$[x]_{反}$，如机器字长为n，则反码的定义如下：

$$[x]_{反}=\begin{cases} x & 0 \leqslant x \leqslant 2^{n-1}-1 \\ (2^n-1)-|x| & -(2^{n-1}-1) \leqslant x \leqslant 0 \end{cases}$$

- 一个正数的反码和原码相同；
- 一个负数的反码的符号位与其原码的符号位相同，其余位通过将其原码的数值部分按位求反得到。

2. 反码

例:如当机器字长n＝8时，

$[+0D]_{反}$＝0000 0000 ，　$[-0D]_{反}$＝1111 1111

$[+1D]_{反}$＝0000 0001 ，　$[-1D]_{反}$＝1111 1110

$[+45D]_{反}$＝0010 1101 ，　$[-45D]_{反}$＝1101 0010

$[+127D]_{反}$＝0111 1111 ，$[-127D]_{反}$＝1000 0000

∴ 0 的表示不惟一

2. 反码

- 按照定义，设n为字长，则反码能表示的整数范围是：$-(2^{n-1}-1)\sim+(2^{n-1}-1)$；

例如：

- 8位二进制反码表示的整数范围是

  －127D～+127D；

- 16位二进制反码表示的整数范围是

  －32767D～+32767D。

3. 补码

- 补码表示法：

数x的补码记作$[x]_{补}$，如机器字长为n，则补码的定义如下：

$$[x]_{补}=\begin{cases} x & 0 \leqslant x < 2^{n-1}-1 \\ 2^{n}-|x| & -(2^{n-1}-1) \leqslant x < 0 \end{cases}$$

- 一个正数的补码和反码、原码相同；
- 一个负数的补码的符号位与其原码的符号位相同，其余位可通过将其反码数值部分加1得到。

3. 补码

- 例如当机器字长n＝8时，

$[+0D]_{补}$＝0000 0000 ，　$[-0D]_{补}$＝0000 0000

$[+1D]_{补}$＝0000 0001 ，　$[-1D]_{补}$＝1111 1111

$[+45D]_{补}$＝0010 1101 ，　$[-45D]_{补}$＝1101 0011

$[+127D]_{补}$＝0111 1111 ，　$[-127D]_{补}$＝1000 0001

∴ 0 的表示惟一

3. 补码

- 按照定义，设n为字长，则补码能表示的整数范围是：
  $-2^{n-1} \sim +(2^{n-1}-1)$

  例如：
  - 8位二进制补码表示的整数范围是

    －128D～+127D；
  - 16位二进制补码表示的整数范围是

    －32768D～+32767D。
- 补码比原码、反码所能表示的数的范围大，数0的补码只有一种表示形式，是计算机中采用的带符号数的编码方式

2.2.2带符号二进制数的运算

1. 补码运算规则

符号位参加运算。

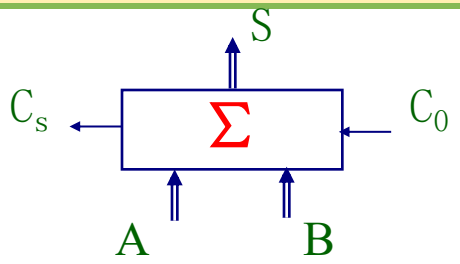
补码运算的两个重要公式：

$[X+Y]_{补} = [X]_{补} + [Y]_{补} \pmod M$

$[X-Y]_{补} = [X]_{补} + [-Y]_{补} \pmod M$ ---- 减法可转化为加法运算！

$= [X]_{补} + [\overline{Y}]_{补} +$ “末位1” $\pmod M$

$S = A + B + C_0$



在计算机中利用加法器就可以实现补码的加法和减法运算。

2. 补码运算的溢出处理

例：设　X=1011B=11D，Y=111B=7D，计算机5位

则　$[X]_{补}$=0,1011，$[Y]_{补}$=0,0111

$$
\begin{array}{rl}
0,1\,0\,1\,1 & [X]_{补} \\
+\ 0,0\,1\,1\,1 & [Y]_{补} \\
\hline
1,0\,0\,1\,0 & [X+Y]_{补}
\end{array}
$$

∴　$[X+Y]_{补}$=1,0010

X+Y=-1110B=-14D

两正数相加结果为-14D，显然是错误的。

例：设 X=-1011B=11D，Y=-111B=7D，**计算机5位**

则 $[X]_{补}$=1,0101，$[Y]_{补}$=1,1001

$$
\begin{array}{rl}
1,0\ 1\ 0\ 1 & [X]_{补} \\
+\quad 1,1\ 0\ 0\ 1 & [Y]_{补} \\
\hline
0,1\ 1\ 1\ 0 & [X+Y]_{补}
\end{array}
$$

∴ $[X+Y]_{补}$=0,1110

X+Y=1110B=14D

两负数相加结果为**14D**，显然也是错误的。

溢出判断与检测方法

1）根据运算前后数的符号位判断

设：操作数 $A = A_s, A_1 A_2 \cdots A_n$

操作数 $B = B_s, B_1 B_2 \cdots B_n$

其和为：$S = S_s, S_1 S_2 \cdots S_n$

则：溢出条件 $V_f = \underline{A_s \cdot B_s} \cdot \overline{S_s} + \underline{\overline{A_s} \cdot \overline{B_s}} \cdot S_s$

两个负数相加，结果却为正数

两个正数相加，结果却为负数

溢出判断与检测方法

2）运算时补码采用双符号位（变形补码）

$$溢出条件V_f = \overline{S_{s1}}\cdot S_{s2} + S_{s1}\cdot\overline{S_{s2}} = S_{s1}\oplus S_{s2}$$

$S_{s1}S_{s2}$=01，结果正溢

$S_{s1}S_{s2}$=10，结果负溢

左边的符号位S_{s1}叫做真符。

当结果的双符号位$S_{s1}S_{s2}$为00或 11时，值用补码能够表示。

例：① A=0.1011，B=-0.1110，求$[A+B]_{补}$。

$$
\begin{array}{rr}
[A]_{补} & 00.1011 \\
+)\ [B]_{补} & 11.0010 \\
\hline
 & 11.1101
\end{array}
$$

$\therefore [A+B]_{补} = 1.1101$

② A=0.1011，B=-0.0010，求$[A-B]_{补}$。

$$
\begin{array}{rr}
[A]_{补} & 00.1011 \\
+)\ [-B]_{补} & 00.0010 \\
\hline
 & 00.1101
\end{array}
$$

$\therefore [A-B]_{补} = 0.1101$

③ A=-0.1101, B=-0.1010，求$[A+B]_{补}$。

$$
\begin{array}{rr}
[A]_{补} & 11.0011 \\
+)\ [B]_{补} & 11.0110 \\
\hline
 & 10.1001
\end{array}
$$

∴$[A+B]_{补}$ 负溢出！

④ A=0.1101, B=-0.1010，求$[A-B]_{补}$。

$$
\begin{array}{rr}
[A]_{补} & 00.1101 \\
+)\ [-B]_{补} & 00.1010 \\
\hline
 & 01.0111
\end{array}
$$

∴$[A-B]_{补}$ 正溢出！

2.3 十进制数的编码

- 四位二进制数来表示一位十进制数，称为二进制编码的十进制数（Binary-Code Decimal），简称BCD码。
- 常见的BCD编码有8421码、2421码、余3码等，8086微型计算机中支持8421BCD码的运算。

1. 常见的BCD编码有8421码、2421码、余3码

十进制数	8421 码	2421 码	余 3 码
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

例：两位十进制数39

可表示为 $(0011\ 1001)_{8421}$ 或 $(0110\ 1100)_{余3码}$

2. 8421BCD码有两种格式

- 压缩8421BCD码（组合8421BCD码）：

  一个字节可以表示两位十进制数。

  例如：96D的表示形式是：

  1001 0110。

- 非压缩8421BCD码（非组合8421BCD码）

  一个字节只表示一位十进制数。

  例如：96D的表示形式是：

  0000 1001 0000 0110。

3. BCD码的加减运算

- 压缩8421BCD码的加减运算
  - 参与运算的操作数为压缩8421BCD码；
  - 结果也是压缩8421BCD码。

【例2-12】16D=00010110，18D=00011000两个数相加。

第一步：做加法

$$
\begin{array}{rll}
 & 0001\ 0110 & \longrightarrow\ 16 \\
+ & 0001\ 1000 & \longrightarrow\ 18 \\
\hline
 & 0010\ 1110 & \longrightarrow\ 2E
\end{array}
$$

【例2-12】

第二步：分析上式运算结果，进行十进制调整。

- 结果分析：

16D＋18D=34D，其中的低位1110不是有效的压缩8421码。

- 原因分析：

运算器仍然进行的是二进制数的运算，采用的进位规则不是十进制运算规定的逢十进一。

- 解决的办法：

将出错的那一位压缩8421BCD码与6相加。

$$
\begin{array}{rl}
 & 0010\ 1110 \\
+ & 0000\ 0110 \quad \longrightarrow \quad \text{个位加6调整} \\
\hline
 & 0011\ 0100 \quad \longrightarrow \quad \text{和=34，正确结果}
\end{array}
$$

【例2-13】

39D=00111001，98D=10011000相加。

解：

0011 1001	→	39
＋ 1001 1000	→	98
1101 0001	→	因为个位向十位有进位，所以需加6调整
＋ 0000 0110		
1101 0111	→	因为十位为1101，它是无效编码，所以需加60调整
＋ 0110 0000		
1 0011 0111	→	和=37，进位=1，结果正确

十位的结果大于9

压缩8421BCD运算的十进制调整规则

- 加法运算后的十进制调整规则
  - 若加法和的个位大于9或向十位有进位，则需要“加6调整”。
  - 若加法和的十位大于9或向百位有进位，则需要“加60调整”。
- 减法运算后的十进制调整规则
  - 若减法差的个位大于9或向十位有借位，则需要“减6调整”。
  - 若减法差的十位大于9或向百位有进位，则需要“减60调整”。

2.4 无符号整数

- 在某些情况下，要处理的数全是正数，此时就没有必要再保留符号位了。可以把最高有效位也作为数值位，这样的数称为无符号数。
  - 8位无符号数的表数范围是：0～255D；
  - 16位无符号数的表数范围是：0～65535D。
- 在计算机中，地址要用无符号数表示。
- 带符号数与无符号数的处理是不一样的，要注意区分。

2.5 字符的表示法

- 在计算机中处理的信息并不全是数，还有字符或字符串。例如，姓名、编号等信息。因此，计算机必须能表示和处理字符。

- 80x86系列计算机中采用美国标准信息交换代码——ASCII（American Standard Code for Information Interchange）来表示一个字符。

	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	、	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	”	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	’	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	¦
1101	CR	GS	—	=	M	]	m	}
1110	SO	RS	•	>	N	↑	n	～
1111	SI	US	/	?	O	_	o	DEL

ASCII

- 标准ASCII码用7位二进制数编码，共有128个。
- 计算机存储器基本单位为8位，ASCII码最高位通常为0，通信时，最高位用作奇偶校验位。
- ASCII码表中的前32个（ 20H前）和最后1个编码是不能显示的控制字符，用于表示某种操作。
- ASCII码表中20H后的94个编码是可显示和打印的字符，其中包括数码0～9，英文字母，标点符号等。
- 字符的ASCII值可以看作字符的码值，如字符“A”的ASCII值为41H，“Z”的ASCII值为5AH，利用这个值的大小可以将字符排序，以后我们会遇到字符串大小比较，实际上是比较ASCII码值的大小。

思考题:1、2、4、5、6、11、12

习题：3、7、8、9、10、13